Haoyang Tang

tanghy@idsse.ac.cn | Sanya, Hainan, China | github.com/wojiaonioo | wojiaonioo.github.io

M.S. student in physical oceanography | Differentiable ocean models, GPU acceleration, variational data assimilation and hybrid modeling

Education

Institute of Deep-sea Science and Engineering, Chinese Academy of Sciences **Physical Oceanography, M.S.** 2025.09 - present

Lanzhou University **Atmospheric Sciences, B.S.** 2019.09 - 2023.06

Project

TorchOcean: a differentiable GPU ocean model (sole developer) 2026.03 - present

Ocean models such as MITgcm are written in Fortran for CPU clusters, and their gradients come from an adjoint generated from the source code, which cannot differentiate neural networks. TorchOcean implements the discretizations of MITgcm in PyTorch so that one GPU program computes forward integrations and their gradients, for 4D-Var data assimilation and for training neural networks inside the model; about 128,000 lines of Python and 15,000 lines of CUDA C++.

Model development and validation

- Regional, global latitude–longitude and cubed-sphere grids; hydrostatic and nonhydrostatic equations; open boundaries, KPP vertical mixing and tidal forcing.
- Instrumented the MITgcm source to compare every stage of a time step: in a 29-day South China Sea run on a 960×912×90 grid, the peak temperature difference from MITgcm is within 3% of the difference between two MITgcm runs that differ only in rounding.

GPU acceleration

- On a 960×960×90 grid, one step on two A800 GPUs went from 786 ms to 142 ms, the step time of MITgcm on about 668 CPU cores; in the nonhydrostatic case two GPUs are 3.4 times faster than MITgcm on 400 cores; one GPU takes 3.1 ns per cell per step in double precision, the same order as the GPU ocean model Oceananigans.
- Restructured the time-step code to remove torch.compile graph fragmentation and per-step recompilation: a step used to break into 131 graphs with 6 functions running uncompiled; the step time fell by 28%.
- Wrote 43 CUDA kernels that each fuse the multi-step computation of KPP mixing, implicit vertical advection, tracer advection or horizontal viscosity; the KPP boundary-layer-depth kernel is more than 80 times faster than the PyTorch version; 6 kernels have hand-written backward passes.
- Fixed a two-GPU deadlock caused by the compiler reordering communication by adding data dependencies to the exchanges; two GPUs are 1.8 times faster than one on the 960×960×90 grid.
- Checkpoints go to GPU or host memory depending on free GPU memory, and the solvers use implicit differentiation: a 100-step gradient on a 480×456×90 grid on one GPU went from 421 s to 30 s, and a gradient costs 3 to 4 times the forward integration.

Variational data assimilation and hybrid modeling

- Solved the convective KPP diffusivity together with temperature and salinity column by column at the new time level and differentiated the solution through the implicit function theorem: the original scheme amplified ten-day-window gradients to 10^{28}; the new directional derivatives agree with finite differences to a relative error of 10^{-6}.
- 4D-Var with the model gradient in South China Sea twin experiments: over 52 one-day windows, the three-day forecast error falls by 31% on average and in every window; one cost-and-gradient evaluation takes 29 s and a 30-iteration window about 15 minutes.
- The same reverse integration trains a network that maps observations and the background to the initial state, with the network parameters in the control vector: on the test windows it closes 86% of the gap in forecast error between the background and 4D-Var, against 77% for a network fitted offline to the 4D-Var analyses, at 1.3% of the 4D-Var cost per window.

Skills

Programming: PyTorch (torch.compile, custom operators, autograd), CUDA C++, Python, Fortran, NCCL / MPI
Scientific computing: ocean numerical models, automatic differentiation and adjoints, variational data assimilation, hybrid modeling
Tools: Git, SLURM, Nsight, Claude Code, Codex